Architectural Integration Framework for Lightspeed Retail X-Series Custom Development

The technical landscape for integrating custom code with Lightspeed Retail X-Series, formerly known as Vend, is characterized by a multi-layered API architecture, a transitionary authentication model, and a robust event-driven framework designed for high-concurrency retail environments.[1] As the retail sector increasingly moves toward a headless or "composable" commerce model, the ability to bridge physical point-of-sale (POS) systems with bespoke web applications, enterprise resource planning (ERP) tools, and specialized business intelligence (BI) engines has become a core requirement for developers.[1] This report provides an exhaustive architectural analysis of the requirements, protocols, and data structures necessary to build and maintain high-performance integrations within the Lightspeed X-Series ecosystem.

Architectural Foundations and API Evolution

The Lightspeed Retail X-Series platform does not utilize a single, unified API. Instead, it operates through a collection of interfaces that reflect the system's long-term evolution and its transition toward a more performant, versioned data model.[3] Understanding the nuances of these versions is the first critical step for any custom implementation.

The Multi-Version Strategy

The current ecosystem is divided into three primary functional versions: API 0.9, API 2.0/2.1, and API 3.0 (Beta). Each version serves a specific operational purpose, and a comprehensive integration will likely require interacting with at least two of these simultaneously.[3]

API Version	Core Purpose	Data Tracking Mechanism	Payload Strategy
API 0.9	Legacy Sale Edits, Webhook Creation, Core Payments	Timestamp-based synchronization	Supports partial payloads for certain legacy endpoints [4]
API 2.0	Products, Inventory, Customers, Consignments	Auto-incrementing version numbers	Optimized, lightweight responses for high-load resilience

			[3]
API 2.1	Refined Product Updates, Variant Creation	Version-based	Targeted field updates for complex product schemas [7]
API 3.0	Modern Pricing, Gift Cards, Search (Beta)	UUID-based and Version-based	Modernized RESTful patterns with improved pagination [4]

The transition from API 0.9 to API 2.0 was fundamentally driven by the need for synchronization precision. In the legacy 0.9 version, synchronization relied on timestamps, which could lead to data collisions or missed updates if multiple changes occurred within the same second—a common occurrence in high-volume retail settings.[3] API 2.0 solved this by introducing auto-incrementing object version numbers, allowing external systems to track every individual change with absolute certainty.[3] For custom code intended to mirror inventory or sales data in an external database, the 2.0 versioning mechanism is an indispensable requirement for maintaining data integrity.

Data Model Inconsistencies and Mapping

A significant challenge for developers is the lack of parity in attribute naming across versions. For example, a sale record retrieved via the API 2.0 GET endpoint will refer to its product array as line_items, whereas the API 0.9 POST endpoint, which is still required for many types of sale modifications, expects that same array to be named register_sale_products.[6] This requires a transformation layer in any custom code to map attributes between the retrieval and update phases of a transaction.[6]

Furthermore, the API 2.0 architecture is intentionally "lighter" than its predecessors, returning less auxiliary or "expanded" data by default.[3] This design reduces network overhead and improves the platform's resilience under high load, but it necessitates that developers make more targeted, granular requests to assemble a complete picture of a retail entity.[3]

Developer Ecosystem and Application Lifecycle

The mechanism by which an integration connects to a Lightspeed Retail account is governed by the developer's registration status and the intended distribution of the custom code.

Organization and App Registration

As of January 2026, Lightspeed has shifted toward a "Private App" model for managing

developer access.[9] This new framework moves away from individual-based credentials and toward organization-owned applications, enhancing security and business continuity.[9]

1. **Developer Portal Registration**: Integration begins with registering as a developer at the official Lightspeed portal. It is important to note that developer credentials are independent of standard Retailer account credentials.[10]
2. **Organization Creation**: Retailers on the Plus plan must now create an "Organization" within their account and invite developers to collaborate.[9] This ensures that the application is owned by the business rather than an individual developer, preventing integration breaks if a specific staff member leaves the company.[9]
3. **Application Approval**: When a developer creates a "Private App" within an organization, it must be approved by the account administrator.[9] Once approved, the developer can generate the necessary tokens to begin interacting with the API.[9]

For public or third-party applications intended for use by multiple retailers, the application must undergo an approval process by Lightspeed.[10] Unapproved apps are restricted to connecting with a maximum of 30 retail stores, which is sufficient for private integrations or staging environments but insufficient for commercial distribution.[10]

Sandbox and Testing Considerations

Lightspeed does not provide dedicated sandbox accounts for API development.[12] Instead, the recommended practice is to create a 14-day trial account for development and testing.[11] Developers can use these trial accounts to explore the API endpoints and test custom logic without impacting live production data. For more permanent testing environments, developers often maintain a dedicated "Test Store" and interact with it using the same OAuth or token mechanisms used in production.[11]

Authentication Architecture and Security

The security protocol for custom code is determined by the application's complexity and the plan level of the retailer. While "Personal Tokens" were historically the standard for simple scripts, the ecosystem is rapidly consolidating around the OAuth 2.0 standard.[9]

OAuth 2.0 Protocol and Token Lifecycle

OAuth 2.0 is the mandatory authorization method for all applications intended for long-term use or multi-retailer deployment.[14] The process follows the standard Authorization Code Grant flow, which ensures that the retailer provides explicit consent for the application to access their data.[14]

OAuth Parameter	Type	Requirement	Description
client_id	String	Mandatory	The unique identifier for your application, generated in the developer portal.[10]
client_secret	String	Mandatory	The secret key used to authenticate your server's requests.[10]
redirect_uri	URL	Mandatory	The callback address on your server that accepts the authorization code.[10]
response_type	String	Mandatory	Must be set to code for the initial request.[10]
state	String	Optional (Recommended)	An opaque, non-guessable string used to prevent Cross-Site Request Forgery (CSRF).[10]
scope	String	Mandatory	A space-delimited list of permissions requested by the app.[14]

The token exchange occurs at the /api/1.0/token endpoint.[10] Upon a successful handshake, the application receives an access_token and a refresh_token. The access token is typically short-lived (often 1 hour or 24 hours depending on the specific implementation) and must be provided in the Authorization: Bearer header of every API request.[10] The refresh_token is used to obtain a new access token without re-prompting the user for authorization. It is vital to securely store and update the refresh token, as each refresh cycle typically invalidates the old one and provides a new one in the response.[14]

Scope-Based Permissions

Modern X-Series integrations use a granular scope model to restrict access to specific features. This "least privilege" approach is a significant security improvement over legacy personal tokens, which essentially granted full administrative access to the retailer's data.[9]

- **sales:read**: Required for retrieving sale history and details.[4]
- **products:write**: Necessary for creating or updating product information.[8]
- **consignments:write:inventory_count**: Required specifically for stocktake operations.[17]
- **webhooks**: Grants permission to create and manage webhook subscriptions.[5]

Developers should only request the minimum set of scopes required for their application's functionality. When the access token is issued, the response will include the scopes actually granted, which must be a subset of those requested.[14]

The Sunset of Personal Tokens

Personal tokens are currently being deprecated and are only available to retailers on the Plus plan.[9] While they offer a quick way to explore the API using tools like Postman, they lack the security controls of OAuth and the new Private App framework.[9] Developers starting new projects are strongly advised to bypass personal tokens and implement OAuth 2.0 from the outset to avoid the need for costly migrations when personal tokens are eventually removed.[9]

Core Entity Management: Products and Inventory

The product data model is the heart of the X-Series system, supporting complex variant structures and multi-location inventory tracking.

Product Hierarchies and Variants

In Lightspeed X-Series, a product can be a standalone item or part of a product family containing multiple variants.[4] A variant parent serves as the container for shared attributes, such as brand or category, while the individual variant children represent the specific permutations of size, color, or material.[7]

When creating a product via the API 2.0 POST /api/2.0/products endpoint, the system expects a structured JSON object.

JSON Object Key	Data Type	Requirement	Description
name	String	Mandatory	The public-facing name of the

			product.[7]
sku	String	Mandatory	The unique Stock Keeping Unit identifier.[7]
handle	String	Optional	The URL-friendly identifier for the product.
price_excluding_tax	Number	Mandatory	The base unit price before taxes are applied.[8]
inventory	Array	Mandatory	An array of objects defining stock levels at various outlets.[8]
product_codes	Array	Optional	Additional identifiers like UPC or EAN.[8]
variants	Array	Optional	The child variant objects if creating a product family.[8]

For variant products, maintaining the variant_parent_id is critical. If custom code updates a variant without correctly referencing its parent, it may inadvertently convert the variant into a standalone product, breaking the organizational structure of the retailer's catalog.[18]

Multi-Outlet Inventory Logic

Inventory in X-Series is not represented as a single global value. Instead, it is a distributed array of records tied to specific "Outlets".[5] To accurately retrieve the stock level for a product, developers must query the specific inventory endpoint: GET /api/2.0/products/{id}/inventory.[4]

The inventory object for an outlet typically includes:

- **count**: The actual quantity on hand.
- **reorder_point**: The minimum stock level before a restock is triggered.
- **restock_level**: The desired quantity after a restock.

Integrations with e-commerce platforms like Shopify or WooCommerce must carefully manage

this multi-outlet logic. The standard recommendation is that the Retail POS serves as the single source of truth for inventory. New products should be created in the POS, and stock levels should push from the POS to the e-commerce engine.[20] When an online sale occurs, the e-commerce platform should push the sale back to the POS, which then automatically decrements the inventory from the designated "online" outlet or warehouse.[18]

The Sales and Transaction Engine

Building code that creates or modifies sales requires a deep understanding of the platform's transaction lifecycle and the mandatory data associations.

Sale States and Attributes

The X-Series has moved away from a simple "status" field in favor of a more nuanced "state" and "attribute" model.[6] The state dictates how the sale impacts inventory and financial reporting.

- **parked**: The sale is saved for later retrieval. Inventory is not yet committed, and the sale is not included in finalized financial reports.[6]
- **pending**: Used for Layby or On-Account sales. The inventory is committed, but the sale is not fully paid.[6]
- **closed**: The transaction is complete. All payments have been received, and inventory is officially decremented.[6]
- **voided**: The sale is cancelled. Developers must never attempt to change the state of a sale once it has been voided, as this can lead to severe corruption of inventory and payment records.[6]

Sales can also have attributes like onaccount, layby, delivery, pickup, or service.[21] These attributes allow the system to trigger specific workflows, such as shipping notifications or service order tracking.[16]

The Critical "Read-Modify-Write" Pattern

As previously noted, the API 0.9 /api/register_sales endpoint—which is the primary interface for creating and editing sales—does not support partial updates (PATCH).[6] It functions as a complete replacement of the sale object. If an integration sends a POST request to update a sale and omits the existing line items or payments, the system will delete those omitted records.[6]

A robust custom integration must follow this specific pattern:

1. Retrieve the full, current sale object using GET /api/2.0/sales/{id}.
2. Apply the necessary changes to the JSON object in the application's memory (e.g., adding a new product or updating a customer association).

3. Perform the version-to-version field transformation (e.g., renaming line_items to register_sale_products).
4. POST the entire, transformed object back to the system.[6]

Taxes, Discounts, and Payments

Taxes in Lightspeed X-Series are applied at the line-item level. To create a valid sale, every product must be associated with a tax_id and a calculated tax amount.[21] For tax-free transactions, developers must use the ID of the "No Tax" record and explicitly set the tax value to zero.[3]

Discounts can be applied either to specific line items (by modifying the price and setting price_set to 1) or to the entire sale.[21] To apply a global discount, developers typically add a special "discount product" to the sale with a negative quantity or a negative price.[22]

Payments are stored in the register_sale_payments array. Each payment object must specify a retailer_payment_type_id, which can be retrieved from the /api/payment_types endpoint.[6] To close a sale, the sum of all payments in this array must equal the total value of the sale (including taxes), and the sale state must be updated to closed.[6]

Event-Driven Development: Webhooks and Remote Rules

To build reactive integrations that respond instantly to retail events, developers must implement webhooks or the more advanced Remote Business Rules.

Webhook Subscription and Payloads

Webhooks allow the Lightspeed platform to push notifications to an external server when specific events occur, such as a new sale being created or a customer being updated.[5]

Webhook Trigger	Event Lifecycle	Common Use Case
sale.created	Instant after POST	Triggering a confirmation email or loyalty update.[18]
customer.updated	Instant after PUT	Syncing profile changes to an external CRM.[18]
consignment.received	Upon stock arrival	Triggering warehouse replenishment or 3PL

		workflows.[18]
register_closure.updated	Upon end-of-day	Automating daily financial reporting in accounting software.[18]

HMAC-SHA256 Signature Verification

Security for webhooks is maintained through a cryptographic signature provided in the X-Signature header of every request.[24] Developers must verify this signature to ensure that the webhook was actually sent by Lightspeed and that the payload has not been intercepted or modified.[24]

The verification process requires:

1. **Raw Payload Capture**: The verification must be performed on the raw, unparsed JSON body of the request.[25]
2. **Secret-Key Hash**: Calculate the HMAC-SHA256 hash of the raw body using the application's client_secret as the key.
3. **Comparison**: Compare the resulting hash with the signature in the header. If they do not match, the request must be discarded.[24]

A critical implementation detail is that any attempt to "beautify" the JSON or parse and re-stringify it before verification will alter the string and cause the verification to fail.[25]

Remote Business Rules for Process Interception

While webhooks are passive (they inform your code *after* something happened), Remote Business Rules allow your custom code to actively intervene *during* a retail process.[23] This feature, available on the Plus plan, allows the POS to pause a transaction and wait for instructions from your server.[23]

For example, when a cashier clicks "Pay" on the sell screen, Lightspeed can trigger a sale.ready_for_payment event to your server.[23] Your server can then respond with specific "Action" payloads:

- **stop**: Prevents the sale from proceeding and displays a custom error message (e.g., "Customer has exceeded their credit limit").
- **confirm**: Displays a modal requiring the cashier to confirm an action (e.g., "This item is non-returnable; have you informed the customer?").
- **set_custom_field**: Automatically populates a hidden metadata field on the sale based on your server's logic.[23]

Customizing the Sell Screen via the Payments API

For developers building custom checkout experiences or specialized payment integrations, the Payments API provides a way to embed custom HTML/JavaScript interfaces directly within the Lightspeed Sell Screen.[26]

postMessage Protocol for In-App Modals

The Payments API utilizes the HTML5 postMessage() standard to communicate between the Lightspeed Sell Screen and the custom code running inside an iframe modal.[26]

1. **Setup**: The custom code can send a SETUP message to toggle the visibility of the modal's close button or configure the initial display.[26]
2. **Data Retrieval**: A DATA message requests the full JSON of the current transaction, allowing the custom code to display exact totals or line-item details to the user.[26]
3. **Action Execution**: After the custom workflow is complete (e.g., a custom loyalty redemption or a third-party payment), the code sends an ACCEPT or DECLINE message back to the Sell Screen.[26]

Enhanced Receipt Injection

A powerful feature of this API is the ability to inject custom HTML content directly into the Lightspeed receipt printing process. Using the receipt_html_extra field in a PRINT or ACCEPT message, developers can add promotional QR codes, specialized warranty information, or digital signatures to the physical or digital receipt.[26]

Payment Message Step	Description	Action and Response
ACCEPT	Finalizes the payment	Completes the sale and triggers the receipt print.[26]
ACCEPT_SIGNATURE	Captures and stores signature	Launches print dialogue and stores signature_base64 data.[26]
EXIT	Clean teardown	Closes all dialogues and unbinds the postMessage handler.[26]

This interface is the only supported way to build deeply integrated front-end extensions that interact with the cashier in real-time at the point of purchase.[26]

Extending Data Entities with Custom Fields

When the standard Lightspeed entities do not support the specific data requirements of a

business—such as storing serial numbers for electronics or fabric types for apparel—developers can utilize the Custom Fields API.[27]

Definition and Permission Scope

Custom fields are first defined at the organization level using the POST /api/2.0/workflows/custom_fields endpoint.[27] A definition specifies the entity it belongs to (e.g., sale, line_item, or customer) and the data type (e.g., string, number).[27]

Once defined, values are assigned via the /api/2.0/workflows/custom_fields/values endpoint.[27] It is an important technical constraint that custom fields are scoped to the application that created them. This prevents multiple third-party integrations from overwriting each other's metadata, but it also means that your custom code will only be able to retrieve and update the fields it explicitly defined.[27]

Operational Reliability and Best Practices

To move custom code from a development trial into a production environment, developers must account for rate limits, error states, and idempotency.

Rate Limit Monitoring and Backoff

Lightspeed enforces rate limits based on the source of the request. Requests originating from an application (e.g., a background sync) are limited separately from those originating from a specific user's interaction.[14]

Applications must monitor the X-RateLimit-Reset and X-RateLimit-Remaining headers in every response.[14] If a 429 Too Many Requests response is received, the application should implement an exponential backoff strategy, waiting until the reset time has passed before retrying the request.[14] The /api/1.0/token endpoint is also independently rate-limited; developers should avoid refreshing tokens more frequently than once per hour to stay within these bounds.[14]

Idempotency for Financial Transactions

For operations that impact a retailer's finances, such as Store Credit or Gift Card transactions, the API supports idempotency via a client_id.[30] This is a unique identifier (typically a UUID) supplied by the developer's code. If a request fails due to a network timeout, the developer can safely retry the request using the same client_id. The Lightspeed system will recognize the ID and ensure that the transaction is only processed once, preventing double-charges or duplicate credits.[30]

Handling API Gotchas and Silent Failures

Developers must be wary of "silent successes" in the X-Series API. Some endpoints, particularly

those in the legacy 0.9 version, may return a 200 OK status even if the payload was partially malformed or if mandatory fields were missing.[21] In these cases, the system might create a "blank" record or ignore the invalid fields without returning an error message. Comprehensive client-side validation and immediate verification of created resources via a subsequent GET request are highly recommended.[21]

Developer Tooling and Community Resources

While many developers choose to build direct REST integrations using standard libraries like cURL, Guzzle (PHP), or Requests (Python), several SDKs and wrappers can accelerate the development process.[7]

SDKs and Wrapper Libraries

Library Name	Language	Status	Key Features
lightspeed_x	Python	Portfolio/Public	Wraps GET, POST, PUT, DELETE; handles auth and formatting.[7]
Apideck PHP SDK	PHP	Production	Type-safe, auto-generated from OpenAPI spec; handles pagination.[31]
lightspeed-xseries-sdk	PHP (Saloon)	Alpha	Community-driven; organized by resource (Products, Sales).[32]
moldersmedia/lightspeedapi	PHP	Stable	Built-in sleep handlers for rate limits; clearer error messages.[29]
darrylmorley/lightspeed-retail-sdk	Node/TS	Active	Comprehensive CLI for token management and database setup.[33]

These libraries can significantly reduce the boilerplate code required for OAuth handshakes and token refreshing, allowing developers to focus on their unique business logic.[33] However, as many are community-driven, developers should thoroughly test them against their specific use cases before deploying to a production environment.[32]

Strategic Conclusion: Building for the Future of Retail

Integrating with Lightspeed Retail X-Series is a multi-dimensional technical challenge that requires a balance between legacy reliability and modern architectural standards. The transition toward API 2.0 and the new Private App framework demonstrates Lightspeed's commitment to a more secure, versioned, and performant ecosystem. For developers, this means that the most successful integrations will be those that embrace OAuth 2.0 early, utilize the auto-incrementing version numbers for perfect synchronization, and leverage the power of Remote Business Rules to create active, value-added POS experiences.

By following the "Read-Modify-Write" pattern for transactions, implementing robust HMAC verification for webhooks, and respecting the distributed nature of multi-outlet inventory, custom code can become a seamless and powerful extension of a retailer's operations. As the platform continues to evolve toward API 3.0, the foundations laid today in clean, version-aware code will ensure that integrations remain stable and scalable in the face of the ever-changing retail landscape.

Works cited

1. Retail APIs for X-Series - Lightspeed, accessed March 10, 2026, https://www.lightspeedhq.com/pos/retail/api/
2. Lightspeed R-Series Vs X-Series Comparison - SKUPlugs, accessed March 10, 2026, https://skuplugs.com/lightspeed-r-series-vs-x-series/
3. Introduction to the Lightspeed Retail (X-Series) API, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/introduction
4. List Sales - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/listsales
5. List webhooks - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/listwebhooks
6. Editing Sales - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/sales_editing_sales
7. nilaster/lightspeed_x: A simple Python wrapper for the Lightspeed X Series API. - GitHub, accessed March 10, 2026, https://github.com/nilaster/lightspeed_x
8. Create product - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/createproduct
9. Understanding private apps and personal tokens in Retail POS (X-Series), accessed March 10, 2026, https://x-series-support.lightspeedhq.com/hc/en-us/articles/25534105904027-Understanding-private-apps-and-personal-tokens-in-Retail-POS-X-Series

10. X-Series API Collection | Documentation | Postman API Network, accessed March 10, 2026, https://www.postman.com/lightspeedhq/x-series-api/documentation/dfu5lxc/x-series-api-collection
11. Quick Start - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/quick_start
12. How can I get a sandbox account for API testing? - Lightspeed Retail (X-Series), accessed March 10, 2026, https://x-series-support.lightspeedhq.com/hc/en-us/articles/25533876082075-How-can-I-get-a-sandbox-account-for-API-testing
13. OAuth 2.0 Process - Lightspeed support, accessed March 10, 2026, https://o-series-support.lightspeedhq.com/hc/en-us/articles/31329293014427-OAuth-2-0-Process
14. Authorization - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/authorization
15. Authorization Overview | Developer Documentation, accessed March 10, 2026, https://api-portal.lsk.lightspeed.app/quick-start/authentication/authorization-overview
16. List Sale Fulfillments - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/getfulfillmentsbysaleid
17. Create a consignment - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/createconsignment
18. Lightspeed Retail POS (X-Series) Webhooks by Zapier Integration - Quick Connect, accessed March 10, 2026, https://zapier.com/apps/lightspeed-retail-pos-x-series/integrations/webhook
19. List product types - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/reference/listproducttypes
20. Information synced between Retail POS (R-Series) and eCom (C-Series), accessed March 10, 2026, https://retail-support.lightspeedhq.com/hc/en-us/articles/229129388-Information-synced-between-Retail-POS-R-Series-and-eCom-C-Series
21. Introduction - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/sales_101
22. Discounts - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/sales_discounts
23. Business Rules - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/workflows_business_rules
24. Webhooks - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/v2026.01/docs/webhooks
25. HMAC Tips for Webhooks - Lightspeed support, accessed March 10, 2026, https://o-series-support.lightspeedhq.com/hc/en-us/articles/31329267751707-HM

AC-Tips-for-Webhooks
26. Reference - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/payments_api_reference
27. Custom Fields - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/docs/workflows_custom_fields
28. 2022-10 Rate Limiting Update - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/changelog/2022-10-rate-limiting-update
29. moldersmedia/lightspeed-api-client - GitHub, accessed March 10, 2026, https://github.com/moldersmedia/lightspeed-api-client
30. Store Credit - Lightspeed Retail (X-Series) API Documentation Hub, accessed March 10, 2026, https://x-series-api.lightspeedhq.com/v2026.01/docs/store_credit
31. Lightspeed PHP SDK - Build Lightspeed Integrations - Apideck, accessed March 10, 2026, https://www.apideck.com/integrations/lightspeed/sdk/php
32. PHP SDK for the LightSpeed X-Series API, generated with Saloon - GitHub, accessed March 10, 2026, https://github.com/dansleboby/lightspeed-xseries-sdk
33. darrylmorley/lightspeed-retail-sdk - GitHub, accessed March 10, 2026, https://github.com/darrylmorley/lightspeed-retail-sdk